Skill

Topic 1 : Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
vennela_07@VennelaPC:~$ mkdir fork_project
vennela_07@VennelaPC:~$ cd fork_project
vennela_07@VennelaPC:~/fork_project$ nano fork_process.c
vennela_07@VennelaPC:~/fork_project$ gcc fork_process.c -o fork_process
vennela_07@VennelaPC:~/fork_project$ ./fork_process
Before fork():
PID: 686, PPID: 645

Parent Process:
PID: 686
PPID: 645
Child PID: 687
State: Waiting for child

Child Process:
PID: 687
PPID: 686
State: Running
Child process finishing.
Child process completed.
Parent process finishing.
vennela_07@VennelaPC:~/fork_project$ ps -o pid,ppid,stat,cmd -C fork_process
    PID    PPID STAT CMD
```

```
 GNU nano 7.2                        fork_process.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Before fork():\n");
    printf("PID: %d, PPID: %d\n", getpid(), getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        printf("\nChild Process:\n");
        printf("PID: %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("State: Running\n");

        sleep(3);

        printf("Child process finishing.\n");
        exit(0);
    }
    else {
        printf("\nParent Process:\n");
        printf("PID: %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("Child PID: %d\n", pid);
        printf("State: Waiting for child\n");
```

```c
        printf("State: Waiting for child\n");

        wait(NULL);

        printf("Child process completed.\n");
        printf("Parent process finishing.\n");
    }

    return 0;
}
```

Topic 2 : Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
vennela_07@VennelaPC:~$ mkdir state_monitor_project
vennela_07@VennelaPC:~$ cd state_monitor_project
vennela_07@VennelaPC:~/state_monitor_project$ nano state_monitor.c
vennela_07@VennelaPC:~/state_monitor_project$ gcc state_monitor.c -o state_monitor
vennela_07@VennelaPC:~/state_monitor_project$ ./state_monitor
Process started
PID: 761

Stage 1: Running (CPU work)

Stage 2: Waiting (sleep)

Stage 3: Waiting for keyboard input
```

```
vennela_07@VennelaPC:~$ ps -o pid,ppid,stat,cmd -p 2450
    PID    PPID STAT CMD
vennela_07@VennelaPC:~$ top -p 2450
top - 04:56:25 up 26 min,  1 user,  load average: 0.07, 0.05, 0
Tasks:   0 total,   0 running,   0 sleeping,   0 stopped,   0 z
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,
MiB Mem :   7767.6 total,   7200.9 free,    502.4 used,    213.
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7265.

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM
```

```
 GNU nano 7.2                  state_monitor.c *

#include <stdio.h>
#include <unistd.h>
#include <time.h>

int main() {
    printf("Process started\n");
    printf("PID: %d\n", getpid());
    fflush(stdout);

    printf("\nStage 1: Running (CPU work)\n");
    fflush(stdout);

    time_t start = time(NULL);
    while (time(NULL) - start < 15) {
        // CPU-intensive work
    }

    printf("\nStage 2: Waiting (sleep)\n");
    fflush(stdout);
    sleep(20);

    printf("\nStage 3: Waiting for keyboard input\n");
    fflush(stdout);
    getchar();

    printf("\nStage 4: Process terminating\n");
    fflush(stdout);

    return 0;
}
```

Topic 3 : Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
vennela_07@VennelaPC:~$ mkdir child_sync_project
vennela_07@VennelaPC:~$ cd child_sync_project
vennela_07@VennelaPC:~/child_sync_project$ nano child_sync.c
vennela_07@VennelaPC:~/child_sync_project$ gcc child_sync.c -o child_sync
vennela_07@VennelaPC:~/child_sync_project$ ./child_sync
Parent process started. PID = 835
Child 1 started. PID = 836
Child 2 started. PID = 837

--- Synchronization using wait() ---
Child 3 started. PID = 838
Child 1 completed. PID = 836
wait(): Child PID 836 finished

--- Synchronization using waitpid() ---
Child 2 completed. PID = 837
waitpid(): Child PID 837 finished
Child 3 completed. PID = 838
waitpid(): Child PID 838 finished

All child processes completed.
Parent process exiting.
```

```
  GNU nano 7.2                  child_sync.c *

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int status;
    int i;

    printf("Parent process started. PID = %d\n", getpid());

    // Create multiple child processes
    for (i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid[i] == 0) {
            printf("Child %d started. PID = %d\n",
                   i + 1, getpid());

            sleep((i + 1) * 2);

            printf("Child %d completed. PID = %d\n",
                   i + 1, getpid());

            exit(i + 1);
        }
    }

    printf("\n--- Synchronization using wait() ---\n");

    // Wait for any child to finish
    pid_t finished = wait(&status);

    if (finished > 0) {
        printf("wait(): Child PID %d finished\n", finished);
    }

    printf("\n--- Synchronization using waitpid() ---\n");

    // Wait for the remaining children in a specific order
    for (i = 0; i < 3; i++) {
        if (pid[i] == finished)
```

```
            continue;

        pid_t result = waitpid(pid[i], &status, 0);

        if (result > 0) {
            printf("waitpid(): Child PID %d finished\n", result>
        }
    }

    printf("\nAll child processes completed.\n");
    printf("Parent process exiting.\n");

    return 0;
}
```

Topic 4 : Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
vennela_07@VennelaPC:~$ mkdir zombie_lab
cd zombie_lab
vennela_07@VennelaPC:~/zombie_lab$ nano zombie.c
vennela_07@VennelaPC:~/zombie_lab$ gcc zombie.c -o zombie
vennela_07@VennelaPC:~/zombie_lab$ ./zombie
Parent process: PID = 898
Child PID = 899
Parent sleeping for 30 seconds.
Check the process table in another terminal.
Child process: PID = 899
Child process terminating.
Parent exiting without collecting child status.
```

2nd terminal

```
vennela_07@VennelaPC:~$ ps -o pid,ppid,stat,cmd -p 5201
    PID    PPID STAT CMD
vennela_07@VennelaPC:~$ grep -E 'Name|State|Pid|PPid' /proc/5201/status
grep: /proc/5201/status: No such file or directory
vennela_07@VennelaPC:~$ nano zombie_fixed.c
vennela_07@VennelaPC:~$ gcc zombie_fixed.c -o zombie_fixed
vennela_07@VennelaPC:~$ ./zombie_fixed
Parent process: PID = 935
Child process: PID = 936
Child PID = 936
Child process terminating.
Parent collected child exit status.
Zombie process eliminated.
vennela_07@VennelaPC:~$ ps -o pid,ppid,stat,cmd -p CHILD_PID
error: process ID list syntax error

Usage:
 ps [options]

 Try 'ps --help <simple|list|output|threads|misc|all>'
  or 'ps --help <s|l|o|t|m|a>'
 for additional help text.

For more details see ps(1).
```

```
 GNU nano 7.2                    zombie.c *

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating.\n");
        exit(0);
    } else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping for 30 seconds.\n");
        printf("Check the process table in another terminal.\n">
```

```
        sleep(30);

        printf("Parent exiting without collecting child status.>
    }

    return 0;
}
```

```
 GNU nano 7.2                zombie_fixed.c *

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    int status;

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating.\n");
        exit(0);
    } else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        waitpid(pid, &status, 0);

        printf("Parent collected child exit status.\n");
        printf("Zombie process eliminated.\n");
    }

    return 0;
}
```